Capítulo 5: Structured Streaming y GraphFrames

Contents

- Structured Streaming
- Procesamiento de grafos con GraphFrames

Structured Streaming

El procesamiento en tiempo real o streaming es el acto de incorporar de manera continua nuevos datos **para ir actualizando el resultado del cálculo**. Lo importante no es simplemente recibir datos en tiempo real sino que, en ese mismo instante, los datos tienen que ser utilizados por la aplicación para hacer un cálculo con ellos, cuyo resultado debe darse también en tiempo real. Los datos de entrada no tienen principio ni fin, y se pueden ver como series de eventos que van llegan a la aplicación y van siendo procesados.

El programa calcula nuevas versiones del resultado según van llegando nuevos datos, de manera que Spark automáticamente actualiza el resultado cuando recibe nuevos datos, sin necesidad del control del usuario.

Casos de uso típicos

Algunos casos de uso típicos en los que se necesita procesar datos en tiempo real son:

- Alertas: un nuevo dato causa que se actualice cierto resultado y a consecuencia, se envíe una alerta a una persona
- Reporting en tiempo real: cuadros de mando construidos sobre agregaciones de datos que se actualizan en tiempo real
- ETL incremental: “mi trabajo batch pero en streaming”. Limpiar y procesar datos en crudo según van llegando, para escribirlos a un data warehouse.

Skip to main content

- Decisiones en tiempo real: ¿es fraudulenta esta transacción? Denegar sobre la marcha en caso de serlo
  - Usando reglas cableadas, o bien usando un modelo de ML
  - Usando un modelo de ML que se actualiza (sin re-entrenar) según llegan nuevos datos (online learning, solo si el modelo lo permite). Es lo menos frecuente al existir pocos algoritmos así, y Spark no los incorpora.

Spark Streaming es el módulo para procesamiento en tiempo real de flujos de datos con Spark. Existen dos APIs distintas para esto.

- La API original que se utilizaba antes de Spark 2 era la API de DStream. Es una API de bajo nivel basada en RDDs y una estructura llamada DStream que es el realidad una colección de RDDs. Proporciona muy poca ayuda a los problemas adicionales que surgen cuando se tienen que procesar datos que van llegando en el tiempo, tales como mantener un estado, procesar datos desordenados, etc.
- La API actual, que se introdujo a partir de Spark 2, se denomina Structured Streaming y es la que se utiliza actualmente. Se basa en un mecanismo que permite aplicar la API estructurada que ya hemos usado (API de Spark SQL y DataFrames) a datos que van llegando en tiempo real, gracias a una abstracción llamada Streaming DataFrames.

Retos de los sistemas de procesamiento real y fuentes de datos típicas

El diseño de sistemas de procesamiento de datos en tiempo real da lugar a una serie de retos propios de este tipo de situaciones:

- Ofrecer baja latencia, es decir, respuesta rápida a un solo evento.
- Ofrecer alto throughput, o en otras palabras, ser capaces de procesar rápidamente un volumen alto de entradas que se han recibido en muy poco tiempo. Se mide en salidas (resultados) por unidad de tiempo. Suele estar en contraposición con una baja latencia.
- Ser capaces de procesar datos desordenados (recibidos en orden diferente al que se generaron)
- Mantener un estado interno para determinar qué está pasando
- Ser capaz de unir datos en streaming con datos batch históricos
- Ofrecer robustez frente a pérdida de datos o a datos que llegan duplicados (se recibe más de una vez el mismo dato), pero sin duplicar salidas.

Las fuentes de datos más habituales de las que se suelen recibir datos en tiempo real

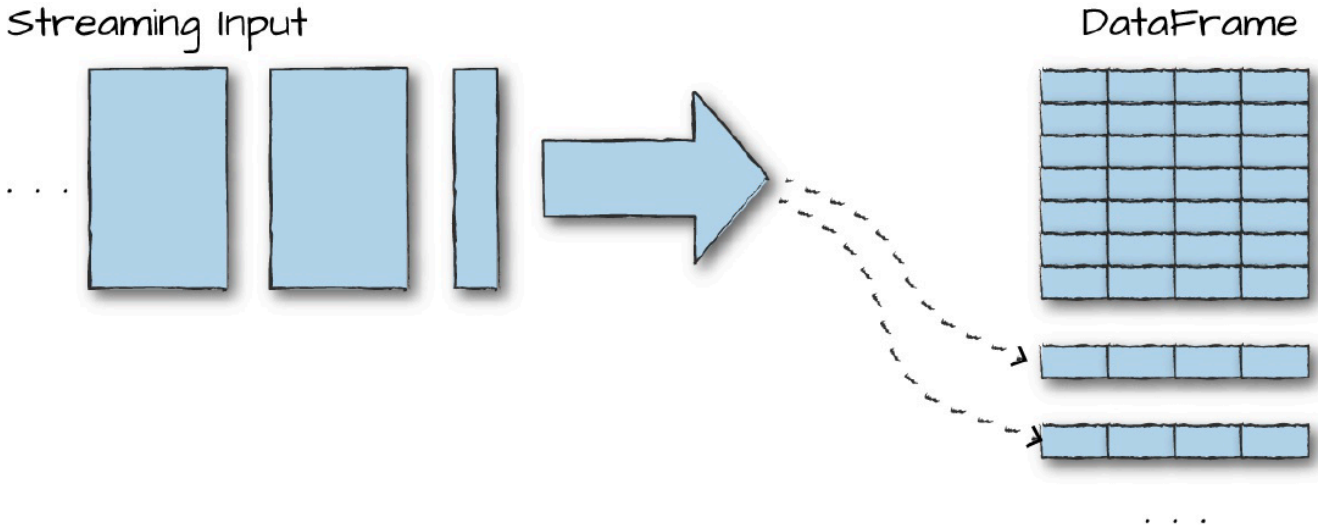
Skip to main content

- Apache Kafka: cola de mensajes distribuida, a la que se le añaden mensajes por un extremo y se consumen por otro. Spark generalmente adopta el rol de consumidor, y asumimos que otros procesos externos son los que están generando en tiempo real nuevos datos y los van escribiendo en Kafka para ser procesados por Spark en algún momento posterior también en tiempo real. Esto es lo más habitual en los sistemas desplegados en producción en la actualidad.
- Sistema de ficheros (DBFS, Azure DataLake Storage, etc): con Structured Streaming es posible monitorizar un directorio de DBFS ó ADLS, en el cual algún otro proceso externo está creando archivos, y del que Spark va leyendo y procesando archivos al mismo tiempo.

La API original de Spark, previa a la versión 2, utilizaba DStreams, que son una secuencia de RDDs. El procesamiento se basa en microbatches: los datos que van llegando se van acumulando en una pequeña ventana (por ejemplo de 500 ms) para formar un pequeño RDD que se procesa en paralelo con Spark, como si fuese un trabajo batch. Esto no es un verdadero sistema de procesamiento en tiempo real, y además tiene más latencia, pero a cambio también mejor throughput. Spark optimiza el throughput en su procesamiento en streaming. Los sistemas que realmente procesan datos de forma continua tienen baja latencia pero mayor sobrecarga por registro, y se acentúa cuando recibimos muchos datos de entrada casi a la vez (suelen tener bajo throughput). En la nueva API de Structured Streaming, seguimos teniendo microbatches, aunque hay una propuesta para soportar verdadero procesamiento continuo, implementada en modo experimental y que obtiene latencias de 1 milisegundo, frente a los 100 milisegundos que ofrece el procesamiento en micro-batches . La idea de Structured Streaming es poder utilizar la misma API de Spark SQL (la API estructurada, basada en DataFrames) a pesar de manejar un DataFrame de tiempo real:

- ¡El mismo código batch que ya teníamos, transformado a streaming!
- Conceptualmente, un Streaming DataFrame es un DataFrame al que se le van añadiendo filas en tiempo real. La filosofía es que el mismo código que ya funcionaba para un trabajo batch debería servir sin cambios en Structured Streaming.

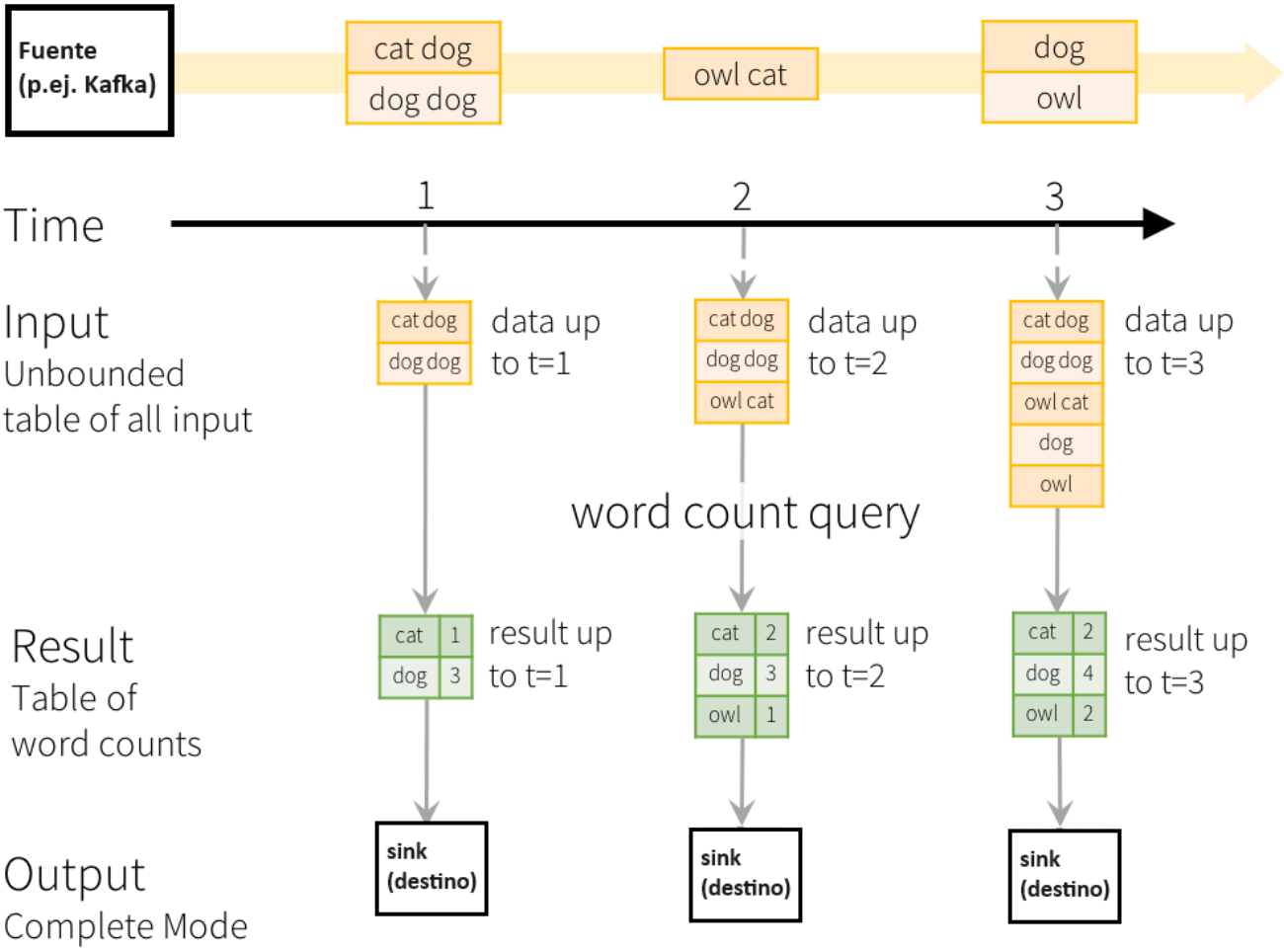
Skip to main content



- Restricción: solo hay una acción disponible sobre un Streaming DataFrame: arrancar un flujo que permanecerá ejecutando continuamente la secuencia de transformaciones que le hayamos indicado, actualizando el resultado continuamente.

La idea para programar con Structured Streaming se muestra en la figura siguiente, mostrando cómo resolveríamos el problema del word count (contar el número de ocurrencias de cada palabra de un DataFrame, como df.groupBy("word").count(), que es un código que podemos ejecutar tal cual sobre un DF batch o sobre un Streaming DF). Habitualmente escribiremos un código que leerá de una fuente (específicamente una que soporte procesamiento en streaming, por ejemplo Kafka aunque existen otras), y obtendremos así un Streaming DataFrame. Le aplicaremos transformaciones mediante la API estructurada, y finalmente el DF resultante, que será también un Streaming DataFrame, lo escribiremos en un destino que soporte también escrituras en tiempo real, conocido genéricamente como "sink". Para ello, tras cada intervalo de ejecución (trigger interval), Spark habrá añadido automáticamente más filas al Streaming DF leído como entrada (un nuevo micro-batch), y estas causarán que se actualice el resultado de la tabla final que estamos guardando. Cada vez que dicha tabla cambia, queremos escribir el resultado actualizado en nuestro almacenamiento de destino. La manera de escribir depende del modo de escritura que hayamos elegido, el cual además depende también del tipo de cálculo que estemos haciendo, ya que no todos los modos de escritura tienen sentido en cualquier cálculo.

Skip to main content



> **Note**
>
> Spark nunca materializa la tabla de entrada completa ni tampoco recalcula usando de nuevo todos los datos desde el principio en cada paso, sino que incrementaliza el cálculo: sólo procesa las filas nuevas, las añade al estado interno (intermedio) que mantenía para esta consulta (secuencia de transformaciones) concreta, y lo usa para actualizar el resultado.

En otras tecnologías el usuario es responsable de mantener estos resultados intermedios para agregarlos con los nuevos datos, pero Spark se ocupa de todo esto, y también de qué ocurre si falla algo durante el procesamiento de un mini-batch de datos.

Modos de escritura

Cuando escribimos resultados en tiempo real, hay varios modos de escritura:

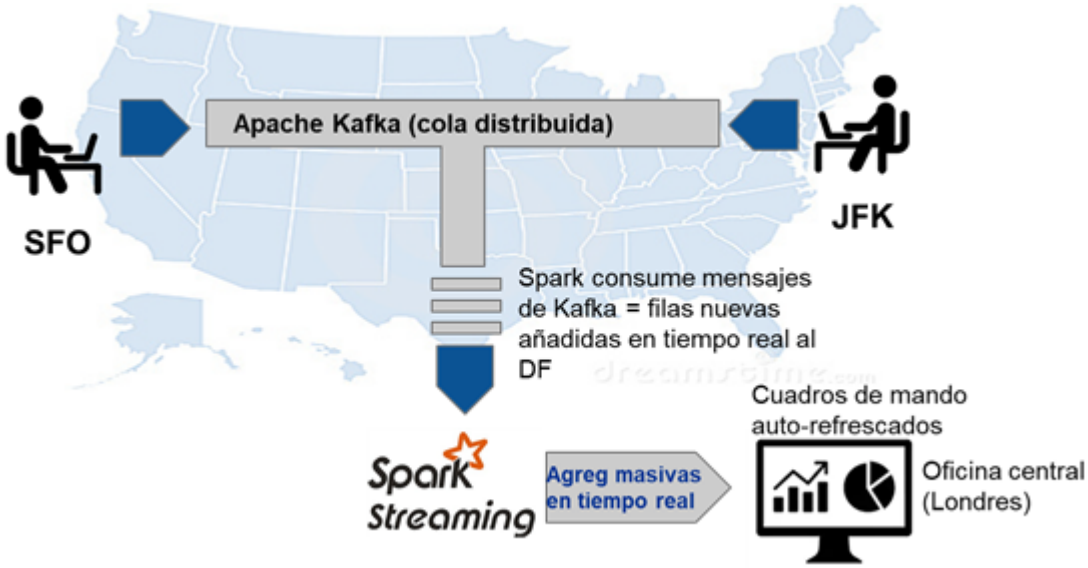
- Añadir (“append”): solo se escriben los nuevos registros al sink de destino. Se debe aplicar cuando las filas ya escritas no van a cambiar, sino que siempre se van a generar

Skip to main content

- Actualizar (“update”): sólo se escribirán los registros que han cambiado desde la escritura anterior (último trigger). Si las transformaciones que estamos haciendo no contienen agregaciones, que provoquen que alguna fila ya calculada pueda haber cambiado, este modo de escritura se comporta igual que append.
- Completo (“complete”): se re-escribe la tabla completa, y depende del almacenamiento destino la manera de decidir cómo almacenar el contenido que ya existía. Adecuado para agregaciones. Dependiendo del tipo de transformación que hagamos, algunos de estos modos no tienen sentido. En nuestro ejemplo de word count, append no tendría sentido porque los resultados de la agregación deben actualizarse, y sigue habiendo el mismo número de registros. En cambio, en una aplicación donde estamos utilizando un modelo predictivo ya entrenado para realizar predicciones sobre nuevos datos que vamos recibiendo en tiempo real, necesitamos append para que cada nueva predicción obtenida sea añadida al resultado, teniendo en cuenta que las predicciones ya realizadas no van a modificarse.

Ejemplo de aplicación al dataset de vuelos

Imaginemos que los aeropuertos envían en tiempo real información a nuestro sistema sobre cada vuelo que aterriza. Se podrían utilizar distintas arquitecturas para esto. La más habitual sería que Spark leyese en tiempo real desde Kafka, una cola de mensajes distribuida y replicada donde operarios que trabajan en los aeropuertos van escribiendo información de vuelos que van aterrizando.



El código para leer un Streaming DataFrame desde Kafka, suscribiéndonos al topic “vuelos”, sería similar a:

Skip to main content

```
vuelos_streaming_df = (
    spark.readStream
         .format("kafka")
         .option("kafka.bootstrap.servers", "hostname.domain:9092")
         .option("subscribe", "vuelos")    # nos suscribimos a todas las particione
         .load()
)
```

A continuación escribimos una secuencia de transformaciones sobre vuelos_streaming_df, el cual es un StreamingDataFrame sobre el que podemos aplicar la API estructurada similar a Spark SQL. Por último, escribimos el resultado de dicha transformación, que es otro streaming DF. Como estamos en modo test, lo escribimos a memoria principal, concretamente a una nueva tabla temporal de Hive, e invocamos sobre él la acción start que desencadena la ejecución. Por último, para que el programa driver no finalice sino que se mantenga esperando indefinidamente, invocamos a awaitTermination. No obstante, en el ejemplo que veremos en el notebook no es necesario este último paso porque el programa driver es el propio notebook, y por tanto se mantendrá corriendo hasta que nosotros cerremos manualmente el kernel del notebook. Podemos ver cómo se va actualizando el resultado en memoria haciendo operaciones de consulta de la tabla resultante en Hive con la operación .sql("consulta"). Si la mostramos varias veces, veremos cada vez un contenido distinto que se va actualizando a medida que Spark va leyendo y procesando nuevos datos de entrada de Kafka.

```
# Operaciones de preprocesamiento como haríamos con cualquier DataFrame
countStreamingDF = vuelos_streaming_df \
                   .withColumn("value", F.col("value").cast("string"))\
                   .withColumn("parsed_flight", F.from_json("value", flights_schem
                   .groupBy("Origin").count()

# Escribimos periódicamente el resultado. Para desarrollos, podemos escribir a memo

countQuery = countStreamingDF\
              .writeStream.queryName("countsPerAirport")\
               .format("memory")\
               .outputMode("complete")\   # Como es una agregación, escribimos la s
               .start()                   # arrancamos el flujo de datos

# lo siguiente es obligatorio para evitar que el driver finalice aunque
# cuando es un notebook, no es necesario
countQuery.awaitTermination()
```

Skip to main content

Watermarks y event-time

Si tenemos en nuestros datos una columna específica (habitualmente de tipo timestamp) del Streaming DataFrame que indica el instante exacto en el cual fue generado ese dato, podemos utilizarla para indicarle a Spark con cuánto retraso (“intervalo de retardo”) esperamos recibir cada dato. Esto le sirve a Spark para poder liberar progresivamente parte de los datos almacenados en su estado interno más antiguo, ya que de otro modo, habría agregaciones (no todas) en las que el estado intermedio que necesita mantener crecería indefinidamente en un proceso que está siempre ejecutándose en producción.

Procesamiento de grafos con GraphFrames

Desde casi el primer momento, Spark posee un módulo para procesar grafos llamado GraphX. Utilizando operaciones distribuidas, implementa algoritmos frecuentes sobre grafos, los cuales están representados mediante RDDs. No obstante, su manejo era tedioso y pronto se vio reemplazado de facto por la biblioteca GraphFrames que estudiamos aquí.

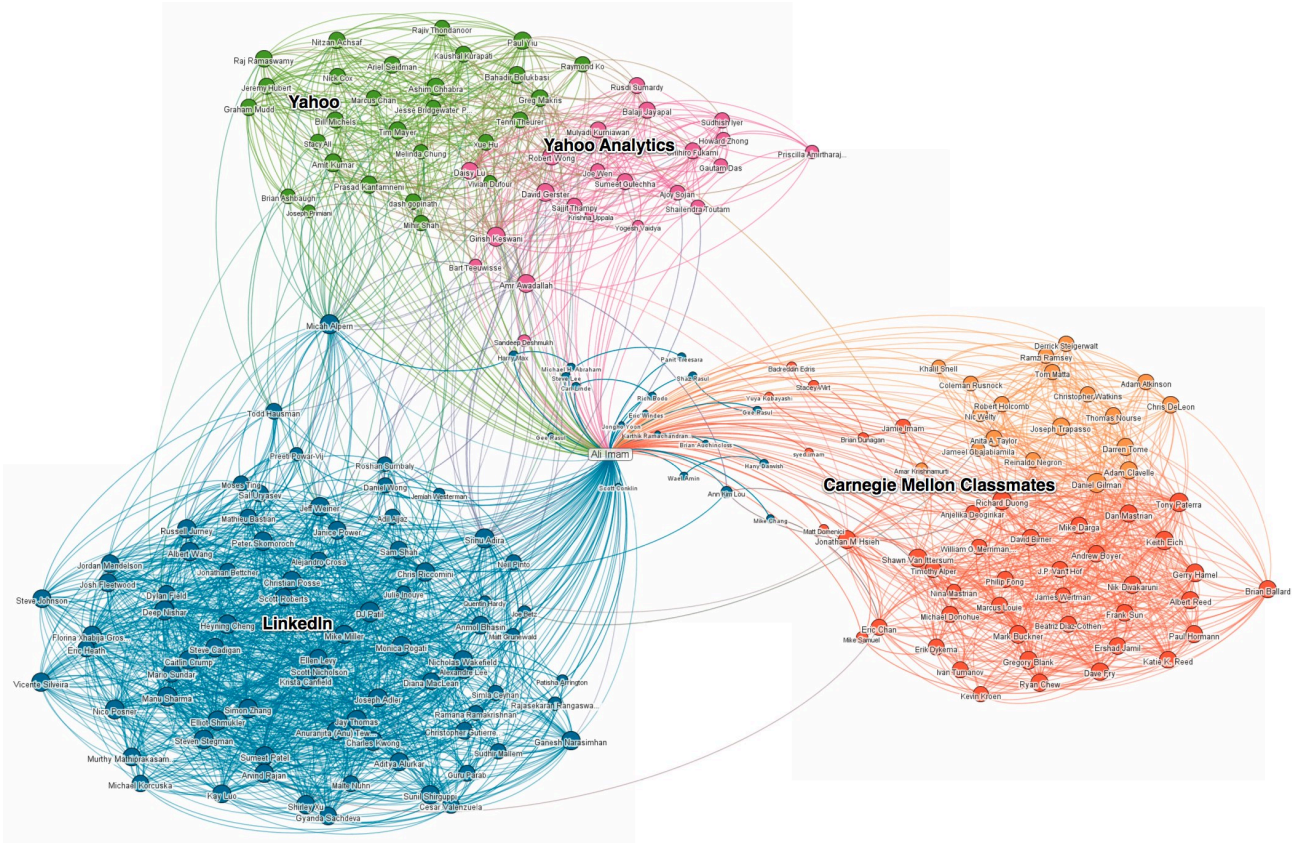
Grafos en la vida real

Un grafo modela relaciones entre objetos en una estructura con vértices (nodos) y arcos (aristas). Los vértices son objetos. Los arcos conectan dos vértices si existe relación entre ellos.

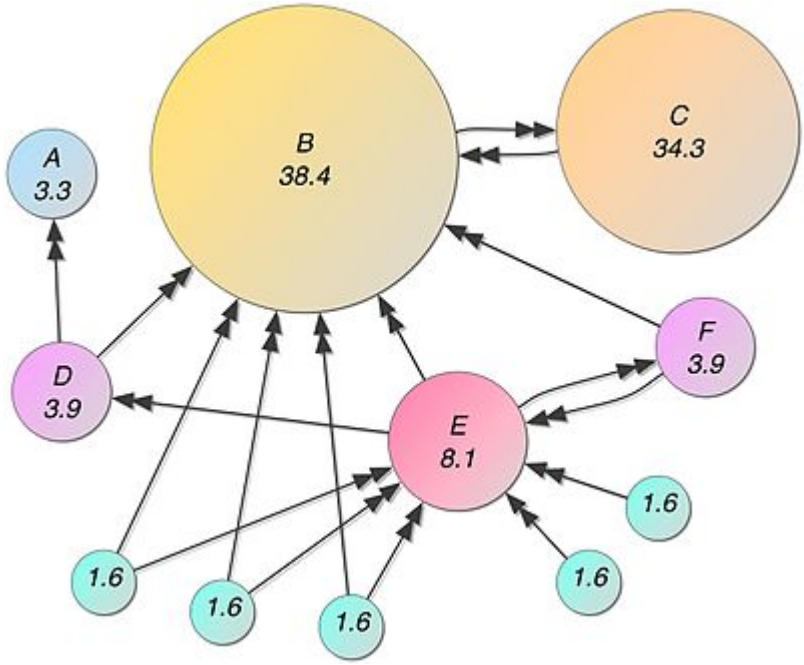
Los nodos y los arcos pueden representar cualquier concepto, y ambos pueden tener propiedades asociadas. El ejemplo perfecto es el de una red social, donde los vértices son personas, y las aristas representan la relación de amistad entre dos personas en la red social. Analizando esta representación en grafo podemos obtener información útil, como por ejemplo:

- Detección de comunidades: muchas relaciones entre sus miembros, pocas con miembros de fuera
- Detección de personas clave (“influencers”) que participan en muchos caminos entre otras dos personas

Skip to main content



Otro ejemplo típico es el de las páginas web en internet. Los vértices son páginas web, y las aristas indican si un sitio web contiene un link que lleva a otro. El algoritmo PageRank de Google mide la importancia de un sitio web como un problema de grafos, en función de cómo está relacionado con los demás, y qué importancia tienen aquellos sitios con los que está relacionado (es decir, sitios que contienen un link a esa web).



Un caso diferente pero muy útil es la identificación (tracking) de dispositivos en la navegación por Internet. Los vértices pueden tener distinta naturaleza: pueden representar una dirección IP, o bien un dispositivo, o bien una cookie. Recordemos que una cookie es un

Skip to main content

identificar visitas recurrentes. Las aristas representan sesiones (cuando un usuario empieza a navegar por nuestra web). El usuario utiliza cierto dispositivo, con cierta IP y con una cookie, que podía existir ya en su dispositivo, o no (si es la primera vez). Las posibles aplicaciones de esta representación se basan en que la misma cookie puede usarse en varias sesiones con distinta IP (por ejemplo si nuestro proveedor de internet ha cambiado la IP, o nos hemos cambiado de compañía), o que un usuario puede usar distintos dispositivos (ordenador portátil, teléfono móvil, Tablet...) desde una misma IP. Al final, un conjunto de vértices interconectados (independientemente de que algunos sean IPs, cookies, etc) representan a la misma persona.

Concepto de grafo

Un grafo es una dupla G = (V, E) siendo V el conjunto de vértices y E el de aristas. Cada vértice puede representar cualquier concepto. Incluso en un mismo grafo pueden convivir vértices que representen cosas distintas (ejemplo de cookies e IPs). Las aristas pueden ser dirigidas o no dirigidas (y así se denomina al grafo). Las aristas pueden tener pesos asociados (ej: distancia, o cualquier otra métrica del grado de intensidad de la conexión). Definimos:

- Grado de entrada de un vértice: número de aristas que llegan al vértice.
- Grado de salida: número de aristas que parten de él.
- Grado de un vértice: suma de ambos grados.

En el dataset de vuelos:

- Vértices: aeropuertos
- Aristas (dirigidas): cuando existe un vuelo directo desde un aeropuerto a otro
- Peso de cada arista: distancia entre los aeropuertos. Podría ser también el número de vuelos diarios entre ellos, o cualquier otra métrica.

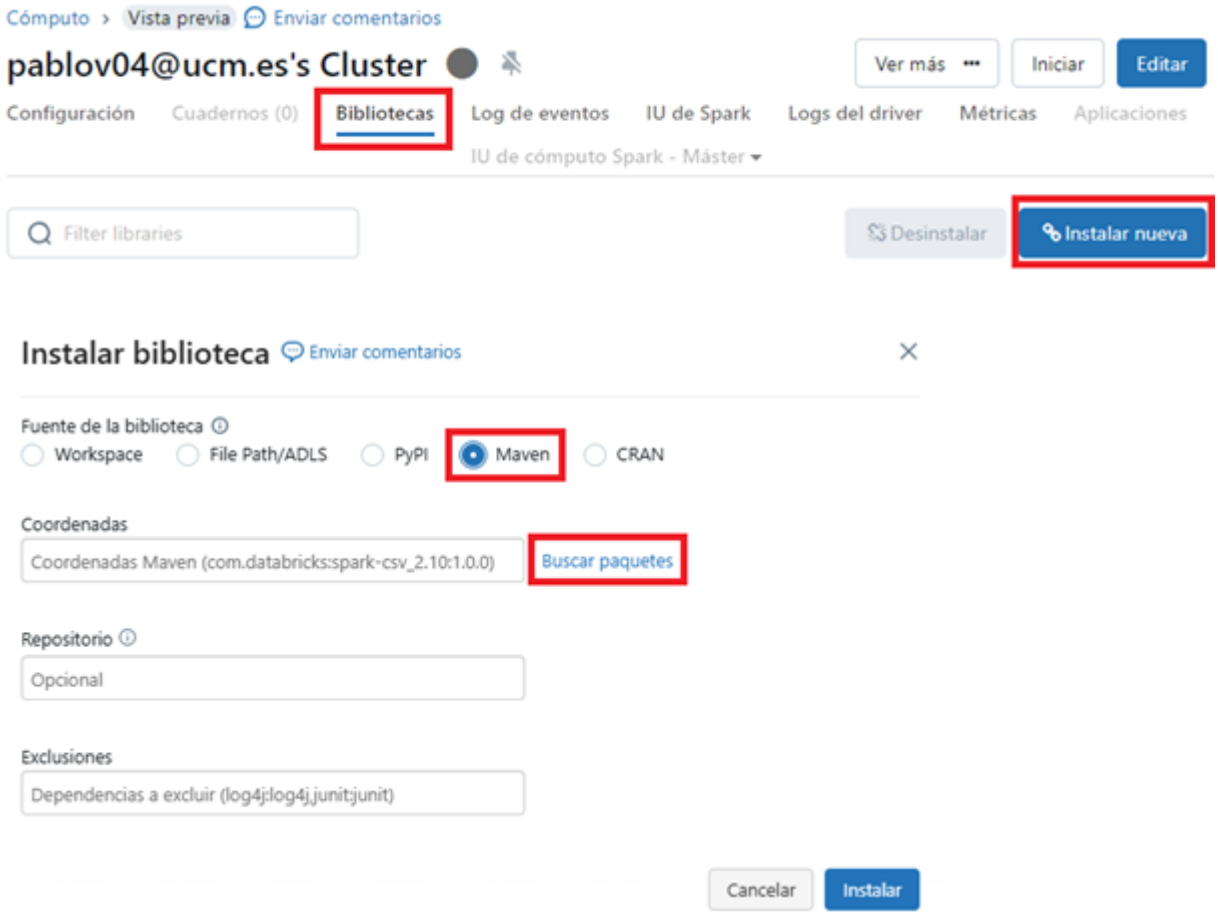
Para calcular, el grafo suele representarse mediante su matriz de adyacencia: una matriz cuadrada de orden igual al número de vértices, donde el valor de la casilla (i, j) puede ser 1 si existe una arista que va desde el vértice i al vértice j o 0 en caso contrario, si el grafo no tiene pesos. Si el grafo tiene pesos, entonces el valor de la casilla (i, j) es igual al peso asociado a esa arista, o infinito si no existe arista.

Skip to main content

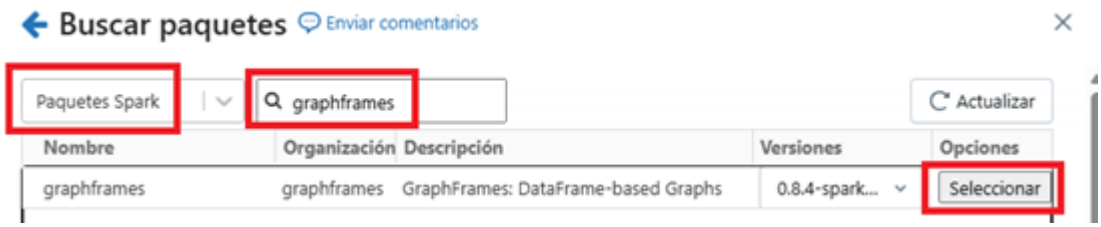
Grafos con Spark

GraphX es el módulo de Spark para procesamiento y algoritmos sobre grafos con RDDs. Forma parte de Spark pero su API es de bajo nivel, y es incómoda de usar, además de no tener optimizaciones en tiempo de ejecución por utilizar RDDs. Puesto que la mayoría de usuarios preferían usar DataFrames, surgió la biblioteca GraphFrames como una evolución de GraphX con DataFrames. Será lo que veremos en este curso. Es un paquete separado que no forma parte de Spark propiamente, por lo que hay que instalarlo expresamente. Está muy extendida y estaba planificado que pasase a formar parte de la descarga de Spark estándar, pero llevan casi dos años sin fusionarlo definitivamente.

GraphFrames está implementado en lenguaje Scala, y por tanto tendremos que instalar un paquete de Scala que se distribuye como un fichero JAR. Después, podemos invocar a dicho código desde Python gracias a un wrapper que se distribuye como un fichero .py con la propia dependencia de Scala (hace un tiempo era necesario instalar por separado un paquete de Python también, pero actualmente no es necesario). En Databricks podemos instalar un paquete en todos los nodos mediante la interfaz gráfica:



Skip to main content



Los paquetes de Java y Scala se pueden instalar de manera automática a partir del repositorio centralizado de paquetes de Maven Central (algo similar al repositorio PyPI que usamos en Python) indicando las coordenadas Maven de la dependencia que queremos instalar. Recordemos que Maven es un software gestor del ciclo de desarrollo y también un repositorio centralizado (Maven Central) de bibliotecas (paquetes en formato JAR) para Java/Scala.

A pesar de haber instalado solamente la dependencia JAR de Scala, podremos invocar al paquete usando las clases de Python porque el wrapper viene incorporado. También es posible utilizar un cluster con DBR (Databricks Runtime) de tipo ML (cualquier versión reciente, por ejemplo DBR 15.4 ML) que trae ya instalada una versión de GraphFrames modificada y optimizada por Databricks.

Grafos con el paquete GraphFrames

En el paquete GraphFrames un grafo se representa mediante dos DataFrames de Spark:

- DataFrame de vértices: debe tener una columna llamada id
- DataFrame de aristas: debe tener columnas llamadas src y dst con los IDs de los vértices que conecten.
- Además de esas columnas obligatorias, pueden existir otras en ambos DF, ya que no intervendrán en los algoritmos que después ejecutaremos sobre el grafo.

Ejemplo:

```
from pyspark.sql import SparkSession, functions as F
from graphframes import GraphFrame

# Opción alternativa para incorporar una dependencia de Java al vuelo, al crear la
# Esto no es necesario en el cluster de Databricks porque ya hemos instalado la dep
# pestaña de Bibliotecas
spark = SparkSession.builder\
                    .config('spark.jars.packages','graphframes:graphframes:0.8.1-sp
                    .getOrCreate()

flightsDF = spark.read.option("header", "true")\
                      .option("inferSchema", "true")\
```

Skip to main content

```
verticesDF = flightsDF.select(F.col("Origin").alias("id"))\
                      .distinct().cache()

edgesDF = flightsDF.withColumnRenamed("Origin", "src")\
                   .withColumnRenamed("Dest", "dst")\
                   .select("src", "dst", "Distance")\
                   .distinct()\
                   .cache()

graph = GraphFrame(verticesDF, edgesDF)
```

```
C:\Users\Lenovo\anaconda3\envs\pyspark\lib\site-packages\pyspark\sql\dataframe.py:1
  warnings.warn(
```

Una vez construido el grafo podemos aplicar distintos algoritmos. Algunos algoritmos devuelven un DataFrame y otros, un grafo completamente nuevo. Grados de entrada y salida: el grafo calcula tres DataFrames:

inDegrees por vértice

- outDegrees por vértice
- degrees por vértice (suma de los grados de entrada y de salida)

```
graph.inDegrees.orderBy(F.col("inDegree").desc()).show(3)

graph.outDegrees.orderBy(F.col("outDegree").desc()).show(3)

graph.degrees.orderBy(F.col("degree").desc()).show(3)
```

```
C:\Users\Lenovo\anaconda3\envs\pyspark\lib\site-packages\pyspark\sql\dataframe.py:1
  warnings.warn("DataFrame constructor is internal. Do not directly use it.")
```

```
+---+--------+
| id|inDegree|
+---+--------+
|ORD|     165|
|ATL|     161|
|DFW|     157|
+---+--------+
only showing top 3 rows
```

```
+---+---------+
| id|outDegree|
```

Skip to main content

```
|ORD|      165|
|ATL|      160|
|DFW|      157|
+---+---------+
only showing top 3 rows
```

```
+---+------+
| id|degree|
+---+------+
|ORD|   330|
|ATL|   321|
|DFW|   314|
+---+------+
only showing top 3 rows
```

Cálculo de componentes conexas:

- Para grafos nos dirigidos: son sub-grafos que están aislados del resto del grafo, es decir, dos vértices cualesquiera están conectados y no es posible viajar fuera de ese subgrafo.
- Para grafos dirigidos, como en el problema de los vuelos, se utiliza el concepto de componentes fuertemente conexas. Son grupos de vértices tales que dos vértices cualesquiera del grupo están conectados. Serían grupos de aeropuertos tales que desde cualquiera del grupo es posible llegar a cualquier otro, directamente o a través de pasos intermedios. En nuestro dataset de vuelos sólo hay una componente conexa (formada por el grafo entero) porque siempre es posible volar de cualquier aeropuerto a cualquier otro, a través de conexiones intermedias con otros aeropuertos.

Este algoritmo devuelve un DataFrame:

```
spark.sparkContext.setCheckpointDir(".")
resultDF = graph.connectedComponents()

resultDF.show(20)    # todos los vértices caen en la misma componente conexa, ya qu

resultDF.groupBy("component").count().show()  # El 0 es un identificador de compone
```

```
+---+---------+
| id|component|
+---+---------+
|ABE|        0|
|ABI|        0|
|ABQ|        0|
|ABR|        0|
|ABY|        0|
```

Skip to main content

```
|ACY|        0|
|ADK|        0|
|ADQ|        0|
|AEX|        0|
|AGS|        0|
|ALB|        0|
|ALO|        0|
|ALW|        0|
|AMA|        0|
|ANC|        0|
|APN|        0|
|ART|        0|
|ASE|        0|
+---+---------+
only showing top 20 rows
```

```
+---------+-----+
|component|count|
+---------+-----+
|        0|  356|
+---------+-----+
```

Algoritmo PageRank

Algoritmo iterativo que utiliza el grado de entrada y de salida de los vértices para determinar la importancia de cada vértice. Es bastante pesado desde el punto de vista computacional. Devuelve un nuevo grafo en el que los vértices tienen una nueva columna de números reales llamada pagerank con la puntuación otorgada por el algoritmo a cada vértice y que indica su importancia. Requiere establecer el checkpointDir de Spark, una carpeta donde cada cierto número de iteraciones se guardan los resultados parciales del algoritmo para casos de fallo.

```
ranksGraph = graph.pageRank(resetProbability=0.15, maxIter=10)
ranksGraph.vertices.orderBy(F.col("pagerank").desc()).show()
```

```
+---+------------------+
| id|          pagerank|
+---+------------------+
|DFW|10.473812993687384|
|ORD|10.260145353782415|
|ATL|  9.43038674195667|
|DEN| 8.853062782143189|
|MSP| 7.554764057993901|
|CLT|   7.2447963343465|
|DTW|6.2044026817052975|
|LAS| 6.105757091251459|
```

Skip to main content

```
|SEA|  5.31619944641921|
|SLC| 5.044415678853287|
|PHL| 4.882291006095359|
|LAX| 4.771622359389784|
|EWR| 4.533074360459349|
|SFO| 4.503367605949667|
|SFB| 4.292855657463282|
|DCA|4.2826697449293665|
|IAD| 3.987501939440854|
|MCO| 3.838361785555211|
+---+------------------+
only showing top 20 rows
```

Algoritmo Breadth-first search

BFS: sirve para calcular el camino más corto entre dos conjuntos de vértices, definido como el más corto en cuanto al número de aristas (no se tienen en cuenta los pesos asociados a una arista). Los conjuntos de partida y llegada los define una expresión booleana. Devuelve un DataFrame como resultado, con una columna que contiene la sucesión de vértices que hay que recorrer para llegar de uno a otro. Todas las columnas son de tipo estructura (struct). En las columnas donde se representan vértices, la estructura contiene como campos anidados las columnas del DataFrame de vértices, que en nuestro caso era solamente el `id`, mientras que en las columnas donde se representan aristas, contiene los 4 campos del DataFrame de aristas (`src`, `dst`, `Distance`).

```
paths = graph.bfs(fromExpr = "id = 'LAX'", toExpr= "id = 'SFO'")
paths.show(truncate=False)
```

```
+-----+-----------------+-----+
|from |e0               |to   |
+-----+-----------------+-----+
|{LAX}|{LAX, SFO, 337.0}|{SFO}|
+-----+-----------------+-----+
```

El paquete GraphFrames no implementa algoritmos de caminos que utilicen los pesos de las artistas, como por ejemplo Dijkstra, Floyd-Warshall, etc. Algoritmo de búsqueda de motivos (motif finding): búsqueda en base a consultas estructurales. Por ejemplo, encontrar dos aeropuertos que no estén directamente conectados pero que se pueda llegar de uno a otro a través de una conexión intermedia. El filtrado posterior evita que nos devuelva todos los aeropuertos ya que nunca es posible ir desde un aeropuerto a sí mismo pero sí a través de cualquier otro aeropuerto intermedio, por lo que en realidad cualquier aeropuerto individualmente cumple la condición respecto de sí mismo. Evidentemente esto no es lo que

Skip to main content

```
res = graph.find("(a)-[]->(b); (b)-[]->(c); !(a)-[]->(c)")\
           .filter("c.id != a.id")

res.show(truncate=False)
```

```
+-----+-----+-----+
|a    |b    |c    |
+-----+-----+-----+
|{ACY}|{ATL}|{LAX}|
|{AEX}|{DFW}|{KOA}|
|{AUS}|{DEN}|{MFR}|
|{AUS}|{MDW}|{PBI}|
|{BDL}|{ORD}|{CAK}|
|{BDL}|{RDU}|{BNA}|
|{BHM}|{MDW}|{BOS}|
|{BIL}|{DEN}|{GSO}|
|{BMI}|{ORD}|{AUS}|
|{BOS}|{MIA}|{BDL}|
|{BOS}|{SEA}|{TUS}|
|{BUF}|{EWR}|{CVG}|
|{BUF}|{ORD}|{MOB}|
|{CAE}|{CLT}|{CHS}|
|{CAK}|{TPA}|{BHM}|
|{CDC}|{SLC}|{MTJ}|
|{CHA}|{ATL}|{MLI}|
|{CHA}|{ORD}|{XNA}|
|{CLT}|{DFW}|{SUX}|
|{CMH}|{ATL}|{PIT}|
+-----+-----+-----+
only showing top 20 rows
```